

Modulo 2:

Normalización de un Sistema de Ventas

Karem Juliet Barandica Arias

Tecnólogo en Análisis y Desarrollo de Software

Jeysson Contreras

Servicio Nacional de Aprendizaje (SENA) Centro de Gestión de Mercados, Logística y
Tecnologías de la Información

19 de junio de 2026

Paso 1. Diagnóstico de la tabla cruda

Identificación de atributos no atómicos

Los siguientes campos contienen múltiples valores separados por comas:

- productos_codigos
- productos_nombres
- categorias
- cantidades
- precios_unitarios
- descuentos

Datos repetidos identificados

Clientes

CC101 – María Gómez

Aparece en:

- V1001
- V1003

Vendedores

VEN01 – Ana Torres

Aparece en:

- V1001
- V1003

Productos

- P001 – Mouse USB
- P002 – Teclado mecánico
- P003 – Monitor 24

Categorías

- Periféricos
- Pantallas
- Computadores
- Accesorios

Métodos de pago

- Transferencia
- Tarjeta de crédito
- Efectivo

Anomalías identificadas

Anomalía de inserción

No es posible registrar un producto nuevo sin crear una venta.

Ejemplo:

- P007 – Impresora

No puede almacenarse independientemente.

Anomalía de actualización

Si cambia el correo de María Gómez:

maria.gomez@mail.com

Debe modificarse en todas las filas donde aparezca.

Si alguna fila no se actualiza, se produce inconsistencia de datos.

Anomalía de eliminación

Si se elimina la venta:

V1004

También se pierde información de:

- Laura Rojas
- Diana Mora
- Webcam HD

Aunque estos datos sigan siendo válidos.

Dependencias funcionales iniciales

venta_id → fecha_venta, cliente_doc, vendedor_id, metodo_pago,
entidad_pago, total_venta

cliente_doc → cliente_nombre, cliente_email,
cliente_telefono, cliente_direccion,
cliente_ciudad

vendedor_id → vendedor_nombre, vendedor_zona

producto_codigo → producto_nombre, categoria,
precio_unitario

categoria → nombre_categoria

Paso 2. Aplicación de Primera Forma Normal (1FN)

Transformación de campos multivaluados

Cada fila debe representar una venta con un único producto asociado.

Tabla ventas_1fn

```
CREATE TABLE ventas_1fn(  
    venta_id VARCHAR(10),  
    fecha_venta DATE,  
    cliente_doc VARCHAR(20),  
    cliente_nombre VARCHAR(100),  
    cliente_email VARCHAR(120),  
    cliente_telefono VARCHAR(30),  
    cliente_direccion VARCHAR(150),  
    cliente_ciudad VARCHAR(80),  
    vendedor_id VARCHAR(10),  
    vendedor_nombre VARCHAR(100),  
    vendedor_zona VARCHAR(80),  
    producto_codigo VARCHAR(10),  
    producto_nombre VARCHAR(100),  
    categoria VARCHAR(100),  
    cantidad INT,  
    precio_unitario NUMERIC(12,2),  
    descuento NUMERIC(12,2),  
    metodo_pago VARCHAR(80),  
    entidad_pago VARCHAR(80),  
    total_venta NUMERIC(12,2)  
);
```

Clave compuesta provisional

(venta_id, producto_codigo)

Paso 3. Aplicación de Segunda Forma Normal (2FN)

Objetivos

- Evaluar el uso de clave compuesta.
- Separar atributos que dependen solo de una parte de la clave.
- Identificar entidades independientes.

Entidades identificadas

- Clientes
- Vendedores
- Productos
- Ventas
- Detalle de ventas

Estructura propuesta

Tabla clientes

```
CREATE TABLE clientes(
  cliente_doc VARCHAR(20) PRIMARY KEY,
  cliente_nombre VARCHAR(100) NOT NULL,
  cliente_email VARCHAR(30) NOT NULL,
  cliente_telefono VARCHAR(100) NOT NULL,
  cliente_direccion VARCHAR(150) NOT NULL,
  cliente_ciudad VARCHAR(80) NOT NULL
);
```

Tabla vendedores

```
CREATE TABLE vendedores(
  vendedor_id VARCHAR(10) PRIMARY KEY,
  vendedor_nombre VARCHAR(100) NOT NULL,
  vendedor_zona VARCHAR(100) NOT NULL
);
```

Tabla productos

```
CREATE TABLE productos(
  producto_codigo VARCHAR(10) PRIMARY KEY,
  producto_nombre VARCHAR(100),
  categoria VARCHAR(100)
);
```

Tabla ventas

```
CREATE TABLE ventas(
  venta_id VARCHAR(10) PRIMARY KEY,
  fecha_venta DATE,
```

```

cliente_doc VARCHAR(20),
vendedor_id VARCHAR(10),
metodo_pago VARCHAR(80),
entidad_pago VARCHAR(80),

CONSTRAINT fk_clientes
    FOREIGN KEY (cliente_doc)
    REFERENCES clientes(cliente_doc),

CONSTRAINT fk_vendedores
    FOREIGN KEY (vendedor_id)
    REFERENCES vendedores(vendedor_id)
);

```

Tabla detalle_ventas

```

CREATE TABLE detalle_ventas(
    venta_id VARCHAR(10),
    producto_codigo VARCHAR(10),
    cantidad INT,
    precio_unitario NUMERIC(12,2),
    descuento NUMERIC(12,2),

    PRIMARY KEY (venta_id, producto_codigo),

    CONSTRAINT fk_ventas
        FOREIGN KEY (venta_id)
        REFERENCES ventas(venta_id),

    CONSTRAINT fk_productos
        FOREIGN KEY (producto_codigo)
        REFERENCES productos(producto_codigo)
);

```

Paso 4. Aplicación de Tercera Forma Normal (3FN)

Objetivos

- Eliminar dependencias transitivas.
- Crear catálogos para categorías, ciudades y zonas.
- Garantizar que cada atributo dependa únicamente de la clave.

Nuevas entidades

- Departamentos

- Ciudades
- Zonas
- Categorías
- Métodos de pago
- Entidades de pago

Relaciones finales

Departamentos → Ciudades

```
CREATE TABLE departamentos(
  id_departamento VARCHAR(10) PRIMARY KEY,
  nombre VARCHAR(100)
);

CREATE TABLE ciudades(
  id_ciudad VARCHAR(10) PRIMARY KEY,
  nombre VARCHAR(100),
  id_departamento VARCHAR(10),

  CONSTRAINT fk_departamentos
    FOREIGN KEY (id_departamento)
    REFERENCES departamentos(id_departamento)
);
```

Clientes

```
CREATE TABLE clientes(
  cliente_doc VARCHAR(20) PRIMARY KEY,
  cliente_nombre VARCHAR(100) NOT NULL,
  cliente_email VARCHAR(30) NOT NULL,
  cliente_telefono VARCHAR(100) NOT NULL,
  cliente_direccion VARCHAR(150) NOT NULL,
  id_ciudad VARCHAR(10) NOT NULL,

  CONSTRAINT fk_ciudades
    FOREIGN KEY (id_ciudad)
    REFERENCES ciudades(id_ciudad)
);
```

Zonas y vendedores

```
CREATE TABLE zonas(
  id_zona VARCHAR(10) PRIMARY KEY,
  zona_nombre VARCHAR(100)
```

);

```
CREATE TABLE vendedores(  
    vendedor_id VARCHAR(10) PRIMARY KEY,  
    vendedor_nombre VARCHAR(100) NOT NULL,  
    id_zona VARCHAR(10) NOT NULL,  
  
    CONSTRAINT fk_zonas  
        FOREIGN KEY (id_zona)  
        REFERENCES zonas(id_zona)  
);
```

Categorías y productos

```
CREATE TABLE categorias(  
    id_categoria VARCHAR(10) PRIMARY KEY,  
    categoria_nombre VARCHAR(100)  
);
```

```
CREATE TABLE productos(  
    producto_codigo VARCHAR(10) PRIMARY KEY,  
    producto_nombre VARCHAR(100),  
    id_categoria VARCHAR(10),  
  
    CONSTRAINT fk_categorias  
        FOREIGN KEY (id_categoria)  
        REFERENCES categorias(id_categoria)  
);
```

Métodos y entidades de pago

```
CREATE TABLE metodos_pago(  
    id_metodo VARCHAR(10) PRIMARY KEY,  
    metodo_nombre VARCHAR(80)  
);
```

```
CREATE TABLE entidades_pago(  
    id_entidad VARCHAR(10) PRIMARY KEY,  
    entidad_nombre VARCHAR(80)  
);
```

Paso 5. Diseño del Modelo Entidad-Relación

Entidades mínimas

- Clientes

- Vendedores
- Ventas
- Detalle_ventas
- Productos
- Categorías

Entidades opcionales

- Métodos de pago
- Entidades de pago
- Ciudades
- Departamentos
- Zonas

Cardinalidades

- Un cliente puede realizar muchas ventas (1:N).
- Un vendedor puede registrar muchas ventas (1:N).
- Una venta puede contener muchos productos (1:N).
- Un producto puede estar presente en muchas ventas (N:M mediante detalle_ventas).
- Una categoría puede contener muchos productos (1:N).

Paso 6. Implementación Física en PostgreSQL

Esquema

CREATE SCHEMA IF NOT EXISTS modulo2_normalizacion;

Restricciones utilizadas

- PRIMARY KEY
- FOREIGN KEY
- NOT NULL
- UNIQUE
- CHECK

Catálogos creados

- Departamentos
- Ciudades
- Zonas
- Categorías
- Métodos de pago
- Entidades de pago

Datos cargados

Se insertaron registros normalizados para:

- Clientes
- Vendedores
- Productos
- Categorías
- Ventas
- Detalle de ventas

Evitando duplicidad de información.

```
CREATE SCHEMA IF NOT EXISTS modulo2_normalizacion;
```

```
CREATE TABLE modulo2_normalizacion.departamentos(
    id_departamento VARCHAR(10) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);
```

```
CREATE TABLE modulo2_normalizacion.ciudades(
    id_ciudad VARCHAR(10) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    id_departamento VARCHAR(10) NOT NULL,
    CONSTRAINT fk_departamentos
        FOREIGN KEY (id_departamento)
        REFERENCES modulo2_normalizacion.departamentos(id_departamento),
```

```

        CONSTRAINT uq_ciudad_departamento
            UNIQUE(nombre, id_departamento)
    );

CREATE TABLE modulo2_normalizacion.clientes(
    cliente_doc VARCHAR(20) PRIMARY KEY,
    cliente_nombre VARCHAR(100) NOT NULL,
    cliente_email VARCHAR(100) NOT NULL UNIQUE,
    cliente_telefono VARCHAR(20) NOT NULL,
    cliente_direccion VARCHAR(150) NOT NULL,
    id_ciudad VARCHAR(10) NOT NULL,
    CONSTRAINT fk_ciudades
        FOREIGN KEY (id_ciudad)
        REFERENCES modulo2_normalizacion.ciudades(id_ciudad)
);

CREATE TABLE modulo2_normalizacion.zonas(
    id_zona VARCHAR(10) PRIMARY KEY,
    zona_nombre VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE modulo2_normalizacion.vendedores(
    vendedor_id VARCHAR(10) PRIMARY KEY,
    vendedor_nombre VARCHAR(100) NOT NULL,
    id_zona VARCHAR(10) NOT NULL,
    CONSTRAINT fk_zonas
        FOREIGN KEY (id_zona)
        REFERENCES modulo2_normalizacion.zonas(id_zona)
);

```

```
CREATE TABLE modulo2_normalizacion.categorias(  
    id_categoria VARCHAR(10) PRIMARY KEY,  
    categoria_nombre VARCHAR(100) NOT NULL UNIQUE  
);
```

```
CREATE TABLE modulo2_normalizacion.productos(  
    producto_codigo VARCHAR(10) PRIMARY KEY,  
    producto_nombre VARCHAR(100) NOT NULL,  
    id_categoria VARCHAR(10) NOT NULL,  
    CONSTRAINT fk_categorias  
        FOREIGN KEY (id_categoria)  
        REFERENCES modulo2_normalizacion.categorias(id_categoria)  
);
```

```
CREATE TABLE modulo2_normalizacion.metodos_pago(  
    id_metodo VARCHAR(10) PRIMARY KEY,  
    metodo_nombre VARCHAR(80) NOT NULL UNIQUE  
);
```

```
CREATE TABLE modulo2_normalizacion.entidades_pago(  
    id_entidad VARCHAR(10) PRIMARY KEY,  
    entidad_nombre VARCHAR(80) NOT NULL UNIQUE  
);
```

```
CREATE TABLE modulo2_normalizacion.ventas(  
    venta_id VARCHAR(10) PRIMARY KEY,  
    fecha_venta DATE NOT NULL,  
    cliente_doc VARCHAR(20) NOT NULL,
```

vendedor_id VARCHAR(10) NOT NULL,

id_metodo VARCHAR(10) NOT NULL,

id_entidad VARCHAR(10) NOT NULL,

CONSTRAINT fk_clientes

FOREIGN KEY (cliente_doc)

REFERENCES modulo2_normalizacion.clientes(cliente_doc),

CONSTRAINT fk_vendedores

FOREIGN KEY (vendedor_id)

REFERENCES modulo2_normalizacion.vendedores(vendedor_id),

CONSTRAINT fk_metodos_pago

FOREIGN KEY (id_metodo)

REFERENCES modulo2_normalizacion.metodos_pago(id_metodo),

CONSTRAINT fk_entidades_pago

FOREIGN KEY (id_entidad)

REFERENCES modulo2_normalizacion.entidades_pago(id_entidad)

);

CREATE TABLE modulo2_normalizacion.detalle_ventas(

venta_id VARCHAR(10),

producto_codigo VARCHAR(10),

cantidad INT NOT NULL,

precio_unitario NUMERIC(12,2) NOT NULL,

descuento NUMERIC(12,2) DEFAULT 0,

PRIMARY KEY (venta_id, producto_codigo),

CONSTRAINT fk_ventas

FOREIGN KEY (venta_id)

REFERENCES modulo2_normalizacion.ventas(venta_id),

CONSTRAINT fk_productos

FOREIGN KEY (producto_codigo)

REFERENCES modulo2_normalizacion.productos(producto_codigo),

CONSTRAINT chk_cantidad

CHECK (cantidad > 0),

CONSTRAINT chk_precio

CHECK (precio_unitario > 0),

CONSTRAINT chk_descuento

CHECK (descuento >= 0)

);

INSERT INTO modulo2_normalizacion.departamentos (id_departamento, nombre)
VALUES

('DEP01','Cundinamarca'),

('DEP02','Antioquia');

INSERT INTO modulo2_normalizacion.ciudades (id_ciudad, nombre, id_departamento)
VALUES

('CIU01','Bogota','DEP01'),

('CIU02','Medellin','DEP02');

INSERT INTO modulo2_normalizacion.clientes (cliente_doc,
cliente_nombre,cliente_email, cliente_telefono, cliente_direccion, id_ciudad) VALUES

('CC101','Maria Gomez','maria.gomez@mail.com','3101112233','Calle 10 # 5-20','CIU01'),

('CC102','Juan Perez','juan.perez@mail.com','3155558899','Carrera 8 # 20-15','CIU01'),

```
('CC103','Laura Rojas','laura.rojas@mail.com','3209994455','Av. 68 # 45-30','CIU02');  
  
INSERT INTO modulo2_normalizacion.zonas (id_zona, zona_nombre) VALUES  
  
('Z01','Norte'),  
  
('Z02','Centro'),  
  
('Z03','Occidente');  
  
INSERT INTO modulo2_normalizacion.vendedores (vendedor_id, vendedor_nombre,  
id_zona) VALUES  
  
('VEN01','Ana Torres','Z01'),  
  
('VEN02','Carlos Ruiz','Z02'),  
  
('VEN03','Diana Mora','Z03');  
  
INSERT INTO modulo2_normalizacion.categorias (id_categoria, categoria_nombre)  
VALUES  
  
('CAT01','Perifericos'),  
  
('CAT02','Pantallas'),  
  
('CAT03','Computadores'),  
  
('CAT04','Accesorios');  
  
INSERT INTO modulo2_normalizacion.productos (producto_codigo, producto_nombre,  
id_categoria) VALUES  
  
('P001','Mouse USB','CAT01'),  
  
('P002','Teclado mecanico','CAT01'),  
  
('P003','Monitor 24','CAT02'),  
  
('P004','Portatil 14','CAT03'),  
  
('P005','Base refrigerante','CAT04'),  
  
('P006','Webcam HD','CAT01');  
  
INSERT INTO modulo2_normalizacion.metodos_pago (id_metodo, metodo_nombre)  
VALUES  
  
('MP01','Transferencia'),  
  
('MP02','Tarjeta credito'),  
  
('MP03','Efectivo');
```

```
INSERT INTO modulo2_normalizacion.entidades_pago (id_entidad, entidad_nombre)
VALUES
```

```
('EP01','Bancolombia'),
```

```
('EP02','Visa'),
```

```
('EP03','Caja principal');
```

```
INSERT INTO modulo2_normalizacion.ventas (venta_id, fecha_venta, cliente_doc,
vendedor_id, id_metodo, id_entidad) VALUES
```

```
('V1001','2026-04-01','CC101','VEN01','MP01','EP01'),
```

```
('V1002','2026-04-02','CC102','VEN02','MP02','EP02'),
```

```
('V1003','2026-04-03','CC101','VEN01','MP01','EP01'),
```

```
('V1004','2026-04-04','CC103','VEN03','MP03','EP03');
```

```
INSERT INTO modulo2_normalizacion.detalle_ventas (venta_id, producto_codigo,
cantidad, precio_unitario, descuento) VALUES
```

```
('V1001','P001',2,45000,0),
```

```
('V1001','P002',1,180000,0),
```

```
('V1002','P003',1,720000,20000),
```

```
('V1002','P001',1,45000,0),
```

```
('V1003','P004',1,2450000,50000),
```

```
('V1003','P002',2,180000,0),
```

```
('V1003','P005',1,95000,0),
```

```
('V1004','P003',2,720000,0),
```

```
('V1004','P006',1,150000,10000);
```

Paso 7. Validación mediante consultas SQL

Reconstruye el total de la venta a partir de los detalles.

Productos más vendidos

Calcula la cantidad total vendida por producto.

Muestra el número de ventas realizadas por cada vendedor.

Calcula el valor total vendido por cada vendedor.

Muestra todas las compras realizadas por cada cliente.

Valida que no existan referencias inválidas en detalle_ventas.

-- TOTAL VENTA --

SELECT

v.venta_id,

v.fecha_venta,

c.cliente_nombre,

SUM((dv.cantidad * dv.precio_unitario) - dv.descuento) AS total_venta

FROM modulo2_normalizacion.ventas v

INNER JOIN modulo2_normalizacion.clientes c

ON v.cliente_doc = c.cliente_doc

INNER JOIN modulo2_normalizacion.detalle_ventas dv

ON v.venta_id = dv.venta_id

GROUP BY

v.venta_id,

v.fecha_venta,

c.cliente_nombre

ORDER BY v.venta_id;

-- Productos más vendidos por cantidad --

SELECT

p.producto_codigo,

p.producto_nombre,

```
SUM(dv.cantidad) AS total_vendido
FROM modulo2_normalizacion.productos p
INNER JOIN modulo2_normalizacion.detalle_ventas dv
    ON p.producto_codigo = dv.producto_codigo
GROUP BY
    p.producto_codigo,
    p.producto_nombre
ORDER BY total_vendido DESC;
```

-- Ventas por vendedor--

```
SELECT
    vd.vendedor_id,
    vd.vendedor_nombre,
    COUNT(v.venta_id) AS numero_ventas
FROM modulo2_normalizacion.vendedores vd
INNER JOIN modulo2_normalizacion.ventas v
    ON vd.vendedor_id = v.vendedor_id
GROUP BY
    vd.vendedor_id,
    vd.vendedor_nombre
ORDER BY numero_ventas DESC;
```

-- ventas totales por vendedor --

```
SELECT
```

```

    vd.vendedor_nombre,
    SUM((dv.cantidad * dv.precio_unitario) - dv.descuento) AS total_vendido
FROM modulo2_normalizacion.vendedores vd
INNER JOIN modulo2_normalizacion.ventas v
    ON vd.vendedor_id = v.vendedor_id
INNER JOIN modulo2_normalizacion.detalle_ventas dv
    ON v.venta_id = dv.venta_id
GROUP BY vd.vendedor_nombre
ORDER BY total_vendido DESC;

```

-- Historial de compras por cliente --

```

SELECT
    c.cliente_doc,
    c.cliente_nombre,
    v.venta_id,
    v.fecha_venta,
    SUM((dv.cantidad * dv.precio_unitario) - dv.descuento) AS total_compra
FROM modulo2_normalizacion.clientes c
INNER JOIN modulo2_normalizacion.ventas v
    ON c.cliente_doc = v.cliente_doc
INNER JOIN modulo2_normalizacion.detalle_ventas dv
    ON v.venta_id = dv.venta_id
GROUP BY
    c.cliente_doc,
    c.cliente_nombre,

```

```

v.venta_id,
v.fecha_venta
ORDER BY
c.cliente_nombre,
v.fecha_venta;

```

-- Verificar registros huérfanos en detalle_ventas --

```

SELECT dv.*
FROM modulo2_normalizacion.detalle_ventas dv
LEFT JOIN modulo2_normalizacion.productos p
    ON dv.producto_codigo = p.producto_codigo
WHERE p.producto_codigo IS NULL;

```

-- Ventas inexistentes--

```

SELECT dv.*
FROM modulo2_normalizacion.detalle_ventas dv
LEFT JOIN modulo2_normalizacion.ventas v
    ON dv.venta_id = v.venta_id
WHERE v.venta_id IS NULL;

```

Dependencia funcional	Tipo	Acción de normalización
cliente_doc → cliente_nombre, cliente_email, cliente_telefono, cliente_direccion	Dependencia funcional completa	Crear tabla clientes con cliente_doc como clave primaria

vendedor_id → vendedor_nombre, vendedor_zona	Dependencia funcional completa	Crear tabla vendedores y separar zonas
producto_codigo → producto_nombre, categoría	Dependencia funcional completa	Crear tabla productos y tabla categorías
venta_id → fecha_venta, cliente_doc, vendedor_id, metodo_pago	Dependencia funcional completa	Crear tabla ventas y catálogos de pago
venta_id + producto_codigo → cantidad, precio_unitario, descuento	Dependencia funcional completa con clave compuesta	Crear tabla detalle_ventas con clave primaria compuesta